

Panduan Lengkap: Memahami Random Forest Classifier

Oleh: Arallyn from herearallyn

Selamat datang di ML Flash with Arallyn! Hari ini kita akan membedah salah satu algoritma paling populer, tangguh, dan serbaguna di dunia AI: **Random Forest Classifier**.

1. Pengertian: Apa itu Random Forest?

Secara sederhana, **Random Forest** adalah algoritma *Machine Learning* yang menggunakan metode **Ensemble Learning** (pembelajaran gabungan).

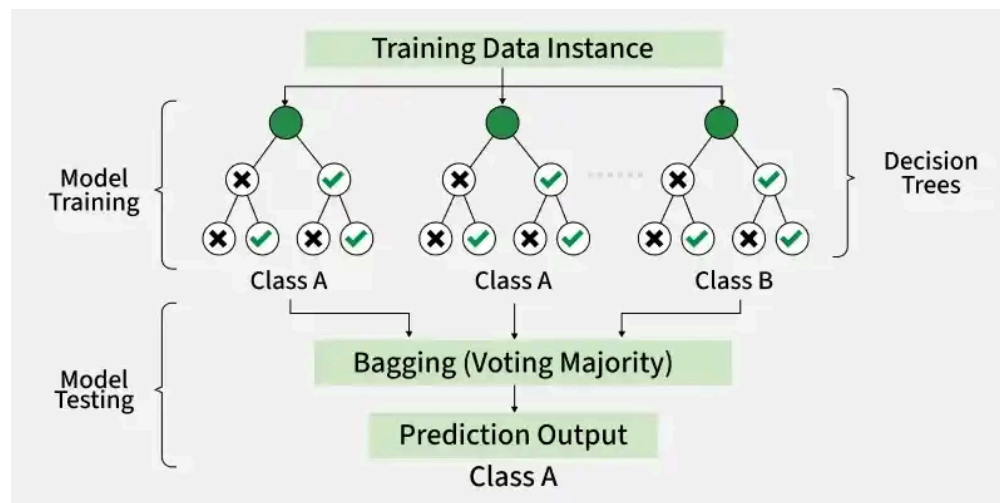
Analogi Sederhana:

Bayangkan Anda ingin membeli mobil baru tapi bingung memilih merek apa.

- Jika Anda bertanya pada **satu orang teman**, keputusannya mungkin sangat bias tergantung selera teman tersebut (Ini adalah **Decision Tree** tunggal).
- Tapi, jika Anda bertanya pada **100 orang teman** yang berbeda latar belakangnya, lalu Anda mengambil suara terbanyak (*voting*) dari jawaban mereka, hasilnya pasti lebih objektif dan akurat.

Nah, **Random Forest** adalah **sekelompok besar Decision Tree (Pohon Keputusan)**.

Masing-masing pohon akan memberikan prediksi, dan Random Forest akan mengambil suara terbanyak (*majority vote*) sebagai hasil akhirnya.



sumber: <https://www.geeksforgeeks.org/random-forest-classifier-using-scikit-learn/>

2. Asal Usul & Sejarah Singkat

Konsep awal Random Forest tidak dibangun dalam semalam:

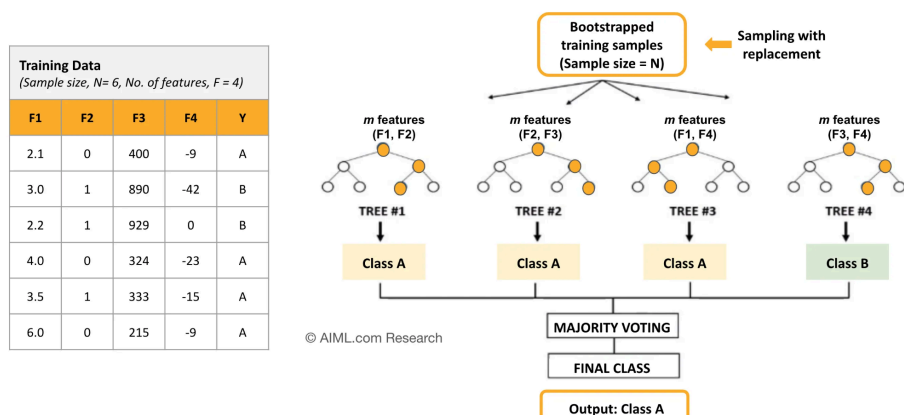
- Pada tahun 1995, **Tin Kam Ho** memperkenalkan metode *Random Subspace* untuk mengatasi kelemahan Decision Tree yang mudah *overfitting* (terlalu menghafal data).
- Kemudian pada tahun 2001, **Leo Breiman** dan **Adele Cutler** menggabungkan ide tersebut dengan teknik *Bagging* (Bootstrap Aggregating) yang **Breiman** ciptakan sebelumnya. Kombinasi inilah yang secara resmi memunculkan algoritma "Random Forest" yang kita kenal sekarang dan telah dipatenkan oleh mereka.

3. Arsitektur: Bagaimana Cara Kerjanya?

Rahasia kekuatan Random Forest terletak pada dua kata: **Acak (Random)** dan **Hutan (Forest)**. Berikut adalah langkah kerjanya:

1. **Bootstrap (Pengambilan Data Acak):** Algoritma tidak memberikan seluruh data yang Anda miliki ke setiap pohon. Ia akan mengambil sampel baris data secara acak (dengan pengembalian/boleh duplikat) untuk membuat dataset baru bagi tiap pohon.
2. **Random Feature Selection (Pemilihan Fitur Acak):** Saat sebuah pohon sedang dibangun dan ingin membelah cabang (membuat aturan), ia tidak melihat *semua* kolom/fitur (misal: umur, gaji, gender). Ia hanya memilih beberapa kolom secara **acak** untuk dipertimbangkan.
3. **Membangun Hutan:** Langkah 1 dan 2 diulang ratusan hingga ribuan kali untuk membangun banyak Decision Tree yang berbeda-beda.
4. **Voting (Prediksi):** Saat ada data baru masuk, data tersebut dilewatkan ke *semua* pohon di dalam hutan. Tiap pohon menebak kelasnya (misal: "Aman" atau "Penipuan"). Random Forest menghitung tebakan terbanyak sebagai jawaban final.

Random Forest Classifier



Key parameters of Random Forest Model are: (a) Number of trees, (b) Maximum depth of the trees (c) Size of the random subset of features. In this example, No. of trees = 4, Depth = 2, and Feature subset size, $m = 2$ (no. of features/2).

source: <https://aiml.com/what-is-random-forest-2/>

4. Kelebihan (Kekuatan Utama)

Mengapa algoritma ini sangat disukai oleh para Data Scientist?

- **Akurasi Sangat Tinggi:** Karena menggabungkan banyak pohon, tingkat kesalahannya jauh lebih rendah dibanding model tunggal.
- **Tahan Terhadap Overfitting:** Berkat teknik pengacakan (*bagging*), Random Forest jarang "menghafal" data latih, sehingga bagus saat memprediksi data baru.
- **Menangani Data yang Hilang (Missing Values):** Bisa tetap bekerja dengan baik meskipun data Anda tidak 100% lengkap.
- **Mengetahui Fitur Terpenting (Feature Importance):** Algoritma ini bisa memberitahu Anda kolom mana yang paling berpengaruh terhadap hasil (misalnya: "Oh, ternyata Umur adalah faktor penentu utama seseorang terkena diabetes").

5. Kekurangan (Sisi Gelap)

Tidak ada algoritma yang sempurna. Ini kelemahannya:

- **Black Box (Sulit Diinterpretasi):** Kalau Anda pakai 1 Decision Tree, Anda bisa melihat alur aturannya ("Jika Umur > 30, maka..."). Tapi kalau ada 500 pohon di Random Forest? Sangat sulit dijelaskan kepada orang awam mengapa model mengambil keputusan tertentu.
- **Komputasi Berat & Makan Memori:** Menyimpan dan melatih ratusan pohon membutuhkan RAM dan daya prosesor (CPU) yang lebih besar.
- **Prediksi Cenderung Lambat:** Karena data baru harus melewati ratusan pohon sebelum diputuskan, model ini kurang cocok untuk sistem *Real-time* yang butuh respon dalam hitungan milidetik.

6. Syarat-syarat Penggunaan

Random Forest sangat *friendly* bagi pemula karena syaratnya tidak banyak:

- **Tidak Butuh Standarisasi/Normalisasi:** Anda tidak perlu repot mengubah skala data (misal dari rentang 10-100.000 menjadi 0-1). Ia mengolah data mentah apa adanya.
- **Menerima Data Campuran:** Bisa menerima kombinasi data angka (numerik) dan kategori (teks/kategorikal) -- *tergantung library yang dipakai, di Scikit-Learn Python, kategori harus di-encode menjadi angka dulu.*
- **Asumsi Independensi:** Fitur atau kolom sebaiknya memiliki daya prediksi (bukan *noise* murni), agar pohon tidak kebingungan membelah cabang.

7. Kapan Memakainya?

Gunakan Random Forest jika Anda menghadapi situasi berikut:

- **Data Tabular:** Data Anda berbentuk tabel (seperti Excel/CSV) yang berisi baris dan kolom.
- **Kompetisi Machine Learning:** Sering dijadikan model dasar (*baseline*) karena tanpa perlu banyak modifikasi (*tuning*), hasilnya sudah lumayan bagus.

- **Kasus Kompleks:** Seperti Deteksi Penipuan Kartu Kredit (*Fraud Detection*), Diagnosa Penyakit Medis, atau Prediksi Churn Pelanggan.
- **Interpretabilitas bukan yang utama:** Anda butuh *akurasi tinggi* lebih dari sekadar *alasan yang mudah dijelaskan*.

8. Perbandingan dengan Algoritma Lain

Kriteria	Random Forest	Decision Tree Tunggal	Logistic Regression	Gradient Boosting (XGBoost/LG BM)
Akurasi	Tinggi	Sedang (Sering Overfit)	Sedang (Hanya pola linier)	Sangat Tinggi
Kecepatan Training	Cepat (Bisa Paralel)	Sangat Cepat	Sangat Cepat	Cenderung Lambat (Sekuensial)
Kemudahan Interpretasi	Sulit (<i>Black Box</i>)	Sangat Mudah (Bisa digambar)	Mudah (Bisa lihat bobot)	Sangat Sulit
Resiko Overfitting	Rendah	Sangat Tinggi	Rendah	Tinggi (Jika tidak di-tuning)
Kebutuhan Tuning	Rendah (<i>Out of the box</i> bagus)	Rendah	Sedang	Sangat Tinggi (Butuh keahlian khusus)

Kesimpulan Perbandingan:

- Jika butuh yang **super cepat dan mudah dijelaskan**, pakai Decision Tree / Logistic Regression.
- Jika butuh **akurasi sangat tinggi tapi Anda ahli men-tuning parameter**, pakai Gradient Boosting.
- Jika butuh **model praktis, akurasi tinggi, stabil, dan minim pusing**, **Random Forest** adalah pemenangnya!

Semoga materi ini membantu Anda memahami Random Forest dengan lebih baik! Selamat mengeksplorasi dunia Machine Learning!

